Handling Multiple Returns

Python returns one object – usually a tuple – that you unpack. Use names once there are many values.

Return a tuple, then unpack it

```
def min_max(nums):
    # a tuple (parentheses optional)
    return min(nums), max(nums)

lo, hi = min_max([4, 9, 2])
print(lo, hi)
# it really is a tuple
print(min_max([4, 9, 2]))
# built-in example
q, r = divmod(17, 5)
print(q, r)
```

OUTPUT

```
2 9
(2, 9)
3 2
```

Unpacking tricks

```
def parts():
    return "a", "b", "c", "d"

# star collects the remainder
first, *rest = parts()
# `_` means "ignore"
_, second, *_ = parts()
print(first, rest, second)
# nested unpacking
(x, y), z = (1, 2), 3
print(x, y, z)
```

OUTPUT

```
a ['b', 'c', 'd'] b
1 2 3
```

Named results are clearer than positions

```
from collections import namedtuple
from dataclasses import dataclass

Stats = namedtuple("Stats", "lo hi")
def spread(nums):
    return Stats(min(nums), max(nums))

s = spread([4, 9, 2])
print(s.lo, s.hi, s)

@dataclass
class Result:
    ok: bool
    value: int = 0

print(Result(True, 5))
```

OUTPUT

```
2 9 Stats(lo=2, hi=9)
Result(ok=True, value=5)
```

Swap and search

```
a, b = 1, 2
# swap: pack, then unpack
a, b = b, a
print(a, b)

def find(items, target):
    for i, x in enumerate(items):
        if x == target:
            # found: (flag, position)
            return True, i
    return False, -1          # not found

print(find("abc", "b"), find("abc", "z"))
```

OUTPUT

```
2 1
(True, 1) (False, -1)
```

Return style	Use when
tuple	2–3 obvious values (`divmod`, `min_max`)
`namedtuple` / `dataclass`	more values, or callers want names
`dict`	dynamic keys / JSON-like data
exception	failure – `raise`, don't return error codes

Several `return` statements & implicit None

```
def sign(n):
    if n < 0:
        return "neg"
    if n == 0:
        return "zero"
    # early returns keep code flat
    return "pos"

def log(msg):
    # no return statement -> None
    print(msg)

print(sign(-3), sign(0), sign(8))
print(log("hi"))
```

OUTPUT

```
neg zero pos
hi
None
```

PITFALL `a, b = f()` fails with `ValueError: too many values to unpack` if `f` returns 3 values. Keep the return shape **consistent** – don't return a number sometimes and a tuple other times.

TIP Add a type hint so callers know the shape: `def min_max(nums: list[int]) -> tuple[int, int]:`